Homotopies and Predictor-Corrector Methods

In this lecture we define homotopies to solve systems based on their total degree, given another application, and explain path following methods. The regularity of the solution paths shows part of a constructive proof of Bézout's theorem. Our treatment follows [4].

1 Homotopies define Deformations

Newton's method is the method of choice to solve nonlinear systems numerically, however: its convergence is only local. To achieve global convergence, we use homotopies. A homotopy is a family of polynomial systems, which connects the system we want to solve $f(\mathbf{x}) = \mathbf{0}$ to a start system $g(\mathbf{x}) = \mathbf{0}$ whose solutions are known. The g stands for $g$ood, because all solutions of $g(\mathbf{x}) = \mathbf{0}$ are well conditioned. A typical homotopy we use is

$$h(\mathbf{x}, t) = \gamma(1-t)g(\mathbf{x}) + tf(\mathbf{x}) = 0, \quad \gamma \in \mathbb{C}, \quad t \in [0,1]. \tag{1}$$

The constant γ is chosen at random to ensure the regularity of the solution paths. Note that a solution of a system $f(\mathbf{x}) = \mathbf{0}$ is regular if and only if its Jacobian matrix evaluated at that solution has full rank. Otherwise, we say that the solution is singular. We will show that for almost all choices of γ, all solutions to $h(\mathbf{x}, t) = \mathbf{0}$ are regular, for all t: $0 \leq t < 1$.

Once the homotopy $h(\mathbf{x}, t) = \mathbf{0}$ is defined, we apply path following methods – also called continuation methods – to track all paths starting at $t = 0$ at the known solutions of $g(\mathbf{x}) = \mathbf{0}$ and ending at $t = 1$, converging to solutions of $f(\mathbf{x}) = \mathbf{0}$. The choice of the start system $g(\mathbf{x}) = \mathbf{0}$ determines the number of solution paths we have to follow and thus the performance of the homotopy method to solve a given polynomial system $f(\mathbf{x}) = \mathbf{0}$.

For systems of polynomials characterized by their degrees, we may use the following start system:

$$g(\mathbf{x}) = \begin{cases} x_1^{d_1} - 1 = 0, & d_1 = \deg(f_1) \\ x_2^{d_2} - 1 = 0, & d_2 = \deg(f_2) \\ \quad \vdots & \quad \vdots \\ x_n^{d_n} - 1 = 0, & d_n = \deg(f_n). \end{cases} \tag{2}$$

We immediately see that the system $g(\mathbf{x}) = \mathbf{0}$ has exactly as many as $D = d_1 \times d_2 \times \cdots \times d_n$ isolated solutions. Since all coordinates of the solutions lie on the unit circle in the complex plane, all solutions are regular.

If we want to solve a system $f(\mathbf{x}) = 0$, we move the parameter t from 0 to 1 and apply Newton's method each time to compute new values along the path, after each update of t. Symbollically, we can let t move in the opposite direction, from 1 to 0, from the given system f to the start system g, interpreting this deformation as a degeneration of f into n univariate equations. Homotopy methods are therefore also related to the so-called method of degeneration in algebraic geometry.

Consider for example the homotopy

$$h(\mathbf{x}, t) = (1-t)\begin{pmatrix} x_1^2 - 1 \\ x_2^2 - 1 \end{pmatrix} + t\begin{pmatrix} x_1^2 + x_2 - 3 \\ x_1 + 0.125x_2^2 - 1.5 \end{pmatrix} = \mathbf{0}. \tag{3}$$

This homotopy defines four solution paths. However, since the choice of γ as one is very specific, the paths run into singularities. In particular for $t \approx 0.92$, the system $h(\mathbf{x}, t) = \mathbf{0}$ has singular solutions. For random complex choices of γ, the four paths in the homotopy converge to the solutions of $h(\mathbf{x}, 1) = \mathbf{0}$.

2 A Problem of Magnetism

Polynomial system often occur in families where the dimension n (the number of equations and variables) varies.

The equations described here appeared in a problem of magnetism in physics. The problem considers the random Ising model (mixture of the ferro- and the antiferro-magnetic bonds) on the infinite Cayley tree of the coordination number z. The distribution function $g(x)(-1 \leq x \geq x)$ of the effective field x is a function of the temperature T, the magnetic field H and the concentration of the ferromagnetic bond p. This distribution function obeys an integral equation defined in [2].

The general formulation of the equations described in [3] is

$$\begin{cases} \sum_{i=-N}^{N} u(i)u(m-i) = u(m) \\ \sum_{i=-N}^{N} u(i) = 1 \end{cases} \tag{4}$$

with $m \in \{-N+1, -N, \ldots, N-1\}$, $u(i) = u(-i)$, and $u(i) = 0$, for $|i| > N$. The number of solutions for a given N is 2^N, equal to the total degree of the system. Solutions of interest are restricted for $u(i) \in [0,1]$.

For $N = 10$, the system

$$\begin{cases} -x_1 + 2x_{11}^2 + 2x_{10}^2 + 2x_9^2 + 2x_8^2 + 2x_7^2 + 2x_6^2 + 2x_5^2 + 2x_4^2 + 2x_3^2 + 2x_2^2 + x_1^2 = 0 \\ -x_2 + 2x_{11}x_{10} + 2x_{10}x_9 + 2x_9x_8 + 2x_8x_7 + 2x_7x_6 + 2x_6x_5 + 2x_5x_4 + 2x_4x_3 + 2x_3x_2 + 2x_2x_1 = 0 \\ -x_3 + 2x_{11}x_9 + 2x_{10}x_8 + 2x_9x_7 + 2x_8x_6 + 2x_7x_5 + 2x_6x_4 + 2x_5x_3 + 2x_4x_2 + 2x_3x_1 + x_2^2 = 0 \\ -x_4 + 2x_{11}x_8 + 2x_{10}x_7 + 2x_9x_6 + 2x_8x_5 + 2x_7x_4 + 2x_6x_3 + 2x_5x_2 + 2x_4x_1 + 2x_3x_2 = 0 \\ -x_5 + 2x_{11}x_7 + 2x_{10}x_6 + 2x_9x_5 + 2x_8x_4 + 2x_7x_3 + 2x_6x_2 + 2x_5x_1 + 2x_4x_2 + x_3^2 = 0 \\ -x_6 + 2x_{11}x_6 + 2x_{10}x_5 + 2x_9x_4 + 2x_8x_3 + 2x_7x_2 + 2x_6x_1 + 2x_5x_2 + 2x_4x_3 = 0 \\ -x_7 + 2x_{11}x_5 + 2x_{10}x_4 + 2x_9x_3 + 2x_8x_2 + 2x_7x_1 + 2x_6x_2 + 2x_5x_3 + x_4^2 = 0 \\ -x_8 + 2x_{11}x_4 + 2x_{10}x_3 + 2x_9x_2 + 2x_8x_1 + 2x_7x_2 + 2x_6x_3 + 2x_5x_4 = 0 \\ -x_9 + 2x_{11}x_3 + 2x_{10}x_2 + 2x_9x_1 + 2x_8x_2 + 2x_7x_3 + 2x_6x_4 + x_5^2 = 0 \\ -x_{10} + 2x_{11}x_2 + 2x_{10}x_1 + 2x_9x_2 + 2x_8x_3 + 2x_7x_4 + 2x_6x_5 = 0 \\ -1 + 2x_{11} + 2x_{10} + 2x_9 + 2x_8 + 2x_7 + 2x_6 + 2x_5 + 2x_4 + 2x_3 + 2x_2 + x_1 = 0 \end{cases} \tag{5}$$

has exactly 1,024 isolated complex solutions.

Even as the number of solutions of this system grows exponentially, we can compute one solution after the other. Since the solution paths can be tracked independently from each other, the speedup on a computer with multiple processors is usually close to optimal.

However, as happens very often with applications, the number of really interesting solutions is quite low. For this application, the number of physically acceptable solutions for a system of dimension N equals one plus the number of divisors of N, see [2] for more details.

3 Predictor-Corrector Methods

To follow the solution paths defined by a homotopy, we apply predictor-corrector methods [1].

Consider a homotopy $h(\mathbf{x}(t), t) = 0$. Since we are interested to see how $\mathbf{x}$ changes as t changes, we apply the operator $\frac{\partial}{\partial t}$ on the homotopy. Via the chain rule, we obtain

$$\frac{\partial h_k}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial t} + \frac{\partial h_k}{\partial t} = 0, \quad \frac{\partial h_k}{\partial \mathbf{x}} = \left[\frac{\partial h_k}{\partial x_1} \frac{\partial h_k}{\partial x_2} \cdots \frac{\partial h_k}{\partial x_n} \right], \quad k = 1, 2, \ldots, n. \tag{6}$$

Denote $\Delta \mathbf{x} := \frac{\partial \mathbf{x}}{\partial t}$. For fixed t (after incrementing $t := t + \Delta t$), we solve the linear system $\frac{\partial h}{\partial \mathbf{x}} \Delta \mathbf{x} = -\frac{\partial h}{\partial t}$ and obtain $\Delta \mathbf{x}$, the tangent to the path. For some step size $\lambda > 0$, the updates $\mathbf{x} := \mathbf{x} + \lambda \Delta \mathbf{x}$ give the Euler predictor.

To avoid solving a linear system at each predictor step, we may use a secant predictor. A secant predictor is less accurate and will require more corrector steps, but the total amount of work for the prediction can be less. Cubic interpolation, using the tangent vectors at two points along the path, leads to the Hermite predictor See Figure 1 for a comparison.

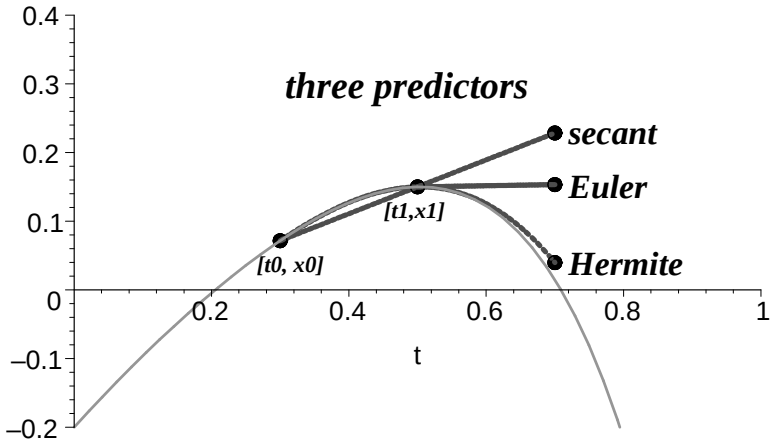


Figure 1: Three predictors: secant, Euler, and Hermite.

The predictor delivers at each step of the method a new value of the continuation parameter and predicts an approximate solution of the corresponding new system in the homotopy. Then, the predicted approximate solution is corrected by applying the corrector, e.g., by Newton's method. With a good homotopy, the solution paths never turn back as t increases. Therefore, the continuation parameter can remain fixed while correcting the predicted solution. This leads to so-called increment-and-fix path following methods. In practice, determining the step length during the prediction stage is done by a hit-or-miss method, which can be implemented by means of an adaptive step size control, as done in Algorithm 3.1.

Algorithm 3.1 Following one solution path by an increment-and-fix predictor-corrector method with an adaptive step size control strategy.

```
Input:  h(x,t), x* ∈ C^n: h(x*,0) = 0,          homotopy and root
        ε > 0, max_it, max_steps,               defines stop criteria
        min_step_size, max_step_size.           for step size control
Output: x*, success if ||h(x*,1)|| ≤ ε.         approximate root at end

t := 0;  k := 0;                                initialization
λ := max_step_size;                             step length
old_t := t;  old_x* := x*                       back up for t and x*
previous_x* := x*;                              previous solution
stop := false;                                  combines stop criteria
while t < 1 and not stop loop
  t := min(1, t + λ);                           secant predictor for t
  x* := x* + λ(x* - previous_x*);               secant predictor for x*
  Newton(h(x,t), x*, ε, max_it,success);        correct with Newton
  if success                                    step size control
   then λ := min(Expand(λ), max_step_size);     enlarge step length
        previous_x* := old_x*;                  go further along path
        old_t := t;  old_x* := x*;              new back up values
   else λ := Shrink(λ);                         reduce step length
        t := old_t;  x* := old_x*;              step back and try again
  end if;
  k := k + 1;                                   augment counter
  stop := (λ < min_step_size)                   1st stop criterion
       or (k > max_steps);                      2nd stop criterion
end loop;
success := (||h(x*,1)|| ≤ ε).                   report success or failure
```

Algorithm 3.1 contains three key ingredients in its loop: the predictor, the corrector and the step size control. The step size λ is controlled by the functions *Shrink* and *Expand* which respectively reduce and enlarge λ, depending on the outcome of the corrector.

The algorithm is still abstract because we did not specify particular values for the constants, such as tolerances on the solutions, minimal and maximal step size, maximum number of iterations of Newton's method, etc.

4 Regularity of the Solution Paths

The following theorem can be seen as a constructive proof for the theorem of Bézout.

Theorem 4.1 *Let $f(\mathbf{x}) = \mathbf{0}$ have total degree D, $g(\mathbf{x}) = \mathbf{0}$ be a start system based on D, and $h(\mathbf{x},t) = \gamma(1-t)g(\mathbf{x}) + t\ f(\mathbf{x}) = \mathbf{0}$, $\gamma \in \mathbb{C}$, $t \in [0,1]$. Except for a set of measure zero, a set of bad choices for γ, the Jacobian matrix of $h(\mathbf{x},t) = \mathbf{0}$ has full rank for t: $0 \leq t < 1$.*

Proof. Denote by J_h the Jacobian matrix of $h(\mathbf{x},t) = \mathbf{0}$ and consider the following extended system:

$$\begin{cases} h(\mathbf{x},t) = \mathbf{0} \\ \det(J_h(\mathbf{x},t)) = 0. \end{cases} \tag{7}$$

The solutions to this extended system characterize those points along the solution paths where the Jacobian matrix drops rank.

If we embed this system in projective space $\mathbb{P}^{n+1}$ we can apply the main theorem of elimination theory n times successively to eliminate all the variables from the system, except for the last variable t. As a result of this elimination, we have one univariate polynomial $p(t)$. This polynomial p vanishes for those t for which the corresponding solution $\mathbf{x}$ is a singular solution of $h(\mathbf{x}, t) = \mathbf{0}$.

Since for $t = 0$, the system $g(\mathbf{x}) = \mathbf{0}$ has only regular solutions, the polynomial p cannot be identically zero. This implies that p has only finitely many solutions and consequently, there are only finitely many complex values for t for which the corresponding $h(\mathbf{x}, t) = \mathbf{0}$ has singular solutions. Observe that for different choices of γ, we have different roots of $p(t)$. In fact, since $t \in [0, 1]$, all roots of $p(t)$ will with probability one miss the segment $0 \leq t < 1$. □

Suppose the given system has a solution of multiplicity m, then systems in the neighborhood will have m regular solutions very close by the multiple root. This argument indicates that, if we apply the homotopy to a system with an m-fold root, then the homotopy will have m paths converging to this multiple root.

The application of the main theorem of elimination theory showed the regularity of the solution paths in a homotopy, but we can also show that all solution paths stay bounded. To that end we formulate the homotopy in projective coordinates and look for solutions for which the added coordinate equals zero. Eliminating all variables except for t we obtain a polynomial where for a random value of the constant γ no roots will lie in $[0, 1]$.

5 using phc and phcpy

`phc` is an executable, available at `http://www.math.uic.edu/~jan/download.html`.

Make a file `example` with content:

```
2
  x^2 + y - 3;
  x + 0.125*y^2 - 1.5;
```

At the command prompt `$`, type

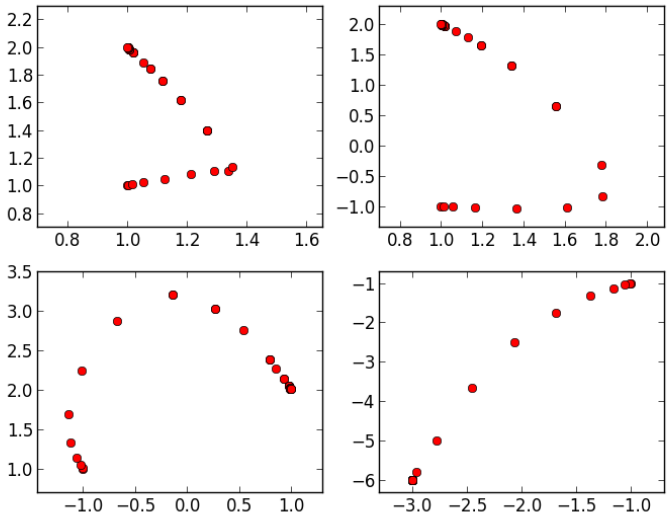
```
$ phc -b example

Do you want the output to file ? (y/n) n
```

Then we see the computed solutions as

```
THE SOLUTIONS :
2 2
=====================================================
solution 1 :
t :  1.00000000000000E+00   0.00000000000000E+00
m : 1
the solution for t :
 x : -3.00000000000000E+00   0.00000000000000E+00
 y : -6.00000000000000E+00   0.00000000000000E+00
= err :  3.4E-16 = rco : 3.2E-01 = res : 4.4E-16 ==
solution 2 :
t :  1.00000000000000E+00   0.00000000000000E+00
m : 3
the solution for t :
 x :  1.00000000000000E+00  -7.33552924616013E-17
 y :  2.00000000000000E+00   1.48991504639649E-16
= err : 1.5E-15 = rco :  1.9E-19 = res : 1.1E-16 ==
```

We recognize $(-3,-6)$ as a solution of multiplicity one, and the solution $(1,2)$ has multiplicity three. The inverse of the condition number `rco` is in the second case very close to zero. The residual `res` is the backward error: the magnitude of what should be added to the system so the computed solution becomes an exact solution of the added system. The `err` is the forward error and gives an upper bound on the error on the coordinates of the computed solution.

With `phcpy` we can make plots of the solution paths:



The Python script is listed below:

```
print 'we fix the seed for reproducibiliy ...'
from phcpy.phcpy2c import py2c_set_seed
py2c_set_seed(34234234)
print 'using the blackbox solver ...'
from phcpy.solver import solve
p = ['x^2 + y - 3;', 'x + 0.125*y^2 - 1.5;']
s = solve(p)
for sol in s:
   print sol
print 'now we redo the computation for one path ...'
print 'constructing a total degree start system ...'
from phcpy.solver import total_degree_start_system as tds
q, qsols = tds(p)
print 'number of start solutions :', len(qsols)
print 'tracking one path ...'
from phcpy.trackers import standard_double_track as track
ss = track(p,q,[qsols[0]])
print ss[0]
print 'deflate the singular solution at the end ...'
from phcpy.solver import deflate
ds = deflate(p,[ss[0]])
print ds[0]
from phcpy.trackers import initialize_standard_tracker
from phcpy.trackers import initialize_standard_solution
from phcpy.trackers import next_standard_solution
initialize_standard_tracker(p,q)
from phcpy.solutions import strsol2dict
```

```
import matplotlib.pyplot as plt
plt.ion()
fig = plt.figure()
for k in range(len(qsols)):
   if(k == 0):
      axs = fig.add_subplot(221)
   elif(k == 1):
      axs = fig.add_subplot(222)
   elif(k == 2):
      axs = fig.add_subplot(223)
   elif(k == 3):
      axs = fig.add_subplot(224)
   startsol = qsols[k]
   initialize_standard_solution(len(p),startsol)
   dictsol = strsol2dict(startsol)
   xpoints =  [dictsol['x']]
   ypoints =  [dictsol['y']]
   for k in range(30):
       ns = next_standard_solution()
       dictsol = strsol2dict(ns)
       xpoints.append(dictsol['x'])
       ypoints.append(dictsol['y'])
   xre = [point.real for point in xpoints]
   yre = [point.real for point in ypoints]
   axs.set_xlim(min(xre)-0.3, max(xre)+0.3)
   axs.set_ylim(min(yre)-0.3, max(yre)+0.3)
   dots, = axs.plot(xre,yre,'ro')
fig.canvas.draw()
ans = raw_input('hit return to exit')
```

6 Exercises

1. Write an algorithm to enumerate all solutions of (2). Modify it so you can compute only the k-th solution, for k any number between 1 and D.

2. For the homotopy (3), use elimination methods to verify that there are singular solutions for $t \approx 0.92$.

3. Consider the homotopy

$$h(x,y,t) = \left(\begin{Bmatrix} x^2 - 1 \\ y^2 - 1 \end{Bmatrix} \right)(1-t) + \left(\begin{Bmatrix} y^2 - 1 \\ x^2 - 3 \end{Bmatrix} \right) t = \mathbf{0}.$$

   For which values of t do we have diverging paths? Show that with a random complex constant γ in $h(x,y,t) = \mathbf{0}$ there are no divergent paths with probability one.

4. Write a Maple procedure or code in Sage to generate the equations defined by (4). Verify the output of your procedure using the system (5).

5. Download the program `phc` (or install `phcpy`) from `http://www.math.uic.edu/~jan/download.html`. Use it to solve the system (5). How many real solutions does the system have?

6. Use `phc` or `phcpy` to track the paths defined by the homotopy (3), once with $\gamma = 1$, and once with a random complex number for γ. Describe what happens.

   The system solved by this homotopy has two distinct roots, one of the roots occurs with multiplicity higher than one. Can you tell from the output which root is multiple?

7. The final observation made in this lecture concerned the number of paths converging to a multiple root. Try to formalize this statement, using a perturbation argument to prove how to count roots with their multiplicities.

References

[1] E.L. Allgower and K. Georg. *Introduction to Numerical Continuation Methods*, volume 45 of *Classics in Applied Mathematics*. SIAM, 2003.

[2] S. Katsura. Users posing problems to PoSSo. In the PoSSo Newsletter, no. 2, July 1994, edited by L. Gonzalez-Vega and T. Recio.

[3] S. Katsura. Spin glass problem by the method of integral equation of the effective field. In M.D. Coutinho-Filho and S.M. Resende, editors, *New Trends in Magnetism*, pages 110–121. World Scientific, 1990.

[4] A.J. Sommese, J. Verschelde, and C.W. Wampler. Introduction to numerical algebraic geometry. In A. Dickenstein and I.Z. Emiris, editors, *Solving Polynomial Equations. Foundations, Algorithms and Applications*, volume 14 of *Algorithms and Computation in Mathematics*, pages 301–337. Springer–Verlag, 2005.